

Superscalar Architecture

Week 6: ROB, CDB, Wakeup/Select — Milestone 3

June 22, 2026

Contents

Week 6 in 60 Seconds	4
1 Week 6 — Milestone 3: Full Out-of-Order Integration	6
2 Comprehensive Theory 1: The Reorder Buffer in Detail [~45 min]	8
2.1 Recap of What the ROB Does	8
2.2 ROB Entry Fields	8
2.3 The Two Pointers and the Count	8
2.4 The Three Operations	9
2.5 The Count Update	9
2.6 Worked Example: A 4-Instruction Trace	10
3 Comprehensive Theory 2: The ALU and the CDB [~30 min]	12
3.1 The ALU	12
3.2 The CDB Concept — and Why We Don’t Need a Real CDB	12
3.3 Why Arbitration Is Needed	13
3.4 The Writeback Connection: $ALU \rightarrow PRF$ and $ALU \rightarrow ROB$	13
3.5 The Wakeup-Select-Issue Loop in Detail	13
3.6 The Bypass Tradeoff (Why We Don’t Implement It)	14
4 Comprehensive Theory 3: Widened Wakeup-Select for 2-ALU Issue [~25 min]	16
4.1 The Naive Approach (and Why It Doesn’t Work)	16
4.2 The Mask-and-Encode Pattern	16
4.3 Concrete Verilog for 4-Entry RS with 2 Issues	16
4.4 Two Issue Buses	17
4.5 Updating RS State	17
4.6 What About Multi-Cycle FUs (Like MUL)?	18
4.7 Same-Cycle Wakeup From Just-Finished FU	18
5 Comprehensive Theory 4: The Top-Level Integration [~20 min]	20
5.1 The Modules in PRAVAH_TOP	20
5.2 The Data Flow Diagram	20
5.3 The Connection Table	20
5.4 Allocate-Ports: Clear PRF Ready on Dispatch	21
5.5 Immediate Operands	22
6 Verilog Implementations [~1 hour reading]	23
6.1 Module: <code>rob.v</code>	23
6.2 Module: <code>alu.v</code>	26
6.3 Module: Top-Level Skeleton	27
6.4 Widening the PRF for 2 Allocate Ports	31

7	The Week 6 Bug-Hunt Checklist [PRINT THIS]	32
7.1	Failure 1: “Nothing happens at all.”	32
7.2	Failure 2: “First instruction dispatches but nothing executes.”	32
7.3	Failure 3: “Lost CDB broadcast” (PRF write or ROB mark-ready missed).	32
7.4	Failure 4: “ROB deadlock” (full ROB; head doesn’t advance).	33
7.5	Failure 5: “Off-by-one in ROB pointer.”	33
7.6	Failure 6: “Stale rename map” (intra-bundle bypass not working).	34
7.7	Failure 7: “Free list corruption.”	34
7.8	The General Debugging Strategy	34
8	The Integration Test — Dot Product [~1 hour]	35
8.1	The Test Program	35
8.2	The Testbench Structure	35
8.3	Running and Verifying	37
8.4	Take Two Waveform Screenshots	37
9	Week 6 Deliverables — Milestone 3	38
A	Appendix: Solutions to Concept Checks	40

Week 6 in 60 Seconds

A one-page summary. **This is the hardest week of the program.** The conceptual content is light — almost everything theoretical was covered in Weeks 1–5. The work is integration: wiring 8+ modules together and getting them to execute a real program.

What you build this week:

- **ROB (rob.v):** a circular buffer of 8 entries, allocated at dispatch in program order, marked ready by FU writebacks, committed in program order at the head.
- **ALU (alu.v):** a single-cycle execution unit that reads operands from the PRF, computes, and writes the result back.
- **CDB arbitration (inside the top module):** when multiple FUs finish in the same cycle, one wins access to the PRF write port and the ROB “mark-ready” port.
- **Wakeup-select extended to 2 ALUs:** the RS module gains a second issue port; two ready RSs can issue per cycle, one to each ALU.
- **Top-level integration (pravah_top.v):** wires everything together. This is where the project becomes real.

The CDB — and why it’s not what classical Tomasulo had:

Classical Tomasulo had a Common Data Bus that broadcast (tag, value) to all RSs. With the PRF design, we don’t broadcast values — they live in the PRF, and RSs hold only indices. What we still need is a **writeback arbitration mechanism**: when multiple FUs finish in the same cycle, only N of them can write to the PRF (where N is the number of PRF write ports, which is 2 in PRAVAH). We call this arbitration logic “the CDB” by convention, even though it doesn’t broadcast values.

The wakeup-select loop is the critical path:

1. Cycle T: ALU finishes, writes to PRF[P_d], sets PRF[P_d].ready.
2. Cycle T (same cycle, combinational): every RS reads its sources’ ready bits. RSs waiting on P_d now see ready=1.
3. Cycle T (same cycle, combinational): priority encoders compute issue_select for each ALU.
4. Cycle T+1: the newly-selected RSs are issued; ALUs begin computing.

This 1-cycle producer-to-consumer latency is what makes the design fast. In a real processor it limits clock frequency; in PRAVAH it’s the most subtle correctness path.

Scope for this week (read carefully):

- Required: ROB, $\text{ALU} \times 2$, CDB arbitration, full integration running an ALU-only test program (dot product).
- **Deferred to Week 7 or stretch:** MUL unit, LSU unit, branch resolution + flush, MUL/LSU reservation stations.

The Milestone 3 gate is “a non-trivial ALU-only program executes correctly out-of-order with in-order commit, end-to-end in ModelSim.” If you finish that, you’ve passed.

Suggested reading order: Section 1 is the ROB’s full design. Section 2 is the ALU and CDB. Section 3 is the widened RS with 2-issue wakeup-select. Section 4 is the top-level integration. Section 5 is the test program. **Section 6 is the bug-hunt checklist — print it and pin it.**

1 Week 6 — Milestone 3: Full Out-of-Order Integration

Goal

A fully integrated PRAVAH pipeline that fetches, decodes, renames, dispatches, executes (ALU-only), and commits a non-trivial ALU-only program. The architectural register file at the end of simulation must match the expected reference values. ModelSim waveforms must show out-of-order execution with in-order commit.

Mentee Work (10 hours)

- Read Sections 1–3 (ROB, ALU, CDB, widened RS, ~2 hours).
- Read Section 4 (top-level integration design, ~30 min).
- Implement `rob.v` with a testbench (~2.5 hours).
- Implement `alu.v` (~30 min — it's the simplest module).
- Widen `reservation_station.v` to support 2-issue (~1.5 hours).
- Build `pravah_top.v` with CDB arbitration (~1.5 hours).
- Write the integration test and run the dot-product program (~2 hours, mostly debugging).

Deliverables — Milestone 3

- `rtl/rob.v` — 8-entry reorder buffer with 2-wide dispatch and 2-wide commit.
- `rtl/alu.v` — single-cycle ALU.
- `rtl/reservation_station.v` — widened to 2-issue (**replaces** Week 5 version).
- `rtl/pravah_top.v` — top-level integration.
- `tb/tb_pravah_top.v` — end-to-end testbench.
- `programs/dot_product.hex` — the benchmark program.
- `sim/waveforms/` — screenshots showing OoO execution + in-order commit.
- Updated `learning_log.md`.

What we Will Check (Milestone 3 Gate)

1. Architectural register file at simulation end matches the expected reference state.
2. Waveform clearly shows: two instructions dispatched in cycle X, two completing out of order in some later cycle, but committing in original program order.
3. No deadlock on full ROB during normal execution.
4. CDB arbitration is deterministic — if two ALUs finish in the same cycle, one consistently wins.
5. Plain Verilog only.

2 Comprehensive Theory 1: The Reorder Buffer in Detail [~45 min]

We have discussed the ROB conceptually since Week 3. Now we build it. This section walks through every operation the ROB supports.

2.1 Recap of What the ROB Does

- **Allocates** entries at dispatch, in program order, two per cycle.
- **Marks entries ready** when their corresponding FU writebacks complete.
- **Commits** (retires) entries from the head, in program order, two per cycle.
- Holds enough information to identify which physical register to free at commit (`old_phys_dest`).

2.2 ROB Entry Fields

Each ROB entry stores:

Field	Purpose
<code>valid</code>	1 = entry is occupied
<code>ready</code>	1 = result has been written; can commit
<code>writes_rd</code>	1 = this inst writes an architectural reg (e.g. ADD, ADDI)
<code>arch_dest</code>	Which arch reg (5 bits for RV32I)
<code>phys_dest</code>	New phys reg holding the result (6 bits for 48-PRF)
<code>old_phys_dest</code>	Phys reg to free at commit

For our subset, we don't carry the PC or exception bits — there are no exceptions in our test programs. (You can add them as a stretch goal in Week 7.)

2.3 The Two Pointers and the Count

- **head**: index of the oldest in-flight instruction. Advances on commit.
- **tail**: index of the next-to-allocate slot. Advances on dispatch.
- **count**: number of valid entries currently in the ROB.

ROB is **full** when `count == ROB_SIZE`. **Empty** when `count == 0`.

We use both a count and pointers (not just pointers) because the empty-vs-full distinction at `head == tail` requires the count to disambiguate.

2.4 The Three Operations

Operation 1: Allocate (at dispatch)

Two ports, each:

- Receive (`arch_dest`, `phys_dest`, `old_phys_dest`, `writes_rd`, `valid`) from dispatch.
- If `valid` and ROB has room: write entry at `tail`. Set `valid=1`, `ready=0`.

Slot A always writes to `tail`; slot B always writes to `tail + 1` (modulo `ROB_SIZE`) *if and only if* slot A also allocated. If only slot A allocates, slot B's port is ignored. (Dispatch handles this stall coordination — the ROB just trusts the inputs.)

The dispatched ROB indices are needed by the RS (so the RS can later tell the ROB which entry to mark ready). The ROB outputs `rob_tail` (the index slot A will use this cycle) as a signal.

Operation 2: Mark Ready (at writeback)

Each FU, when it completes, signals the ROB:

- “Entry at ROB index K is done; set `ready[K] = 1`.”

The ROB has as many mark-ready ports as it has FUs that can complete in parallel. For PRAVAH (2 ALUs), that's 2 ports.

Important: mark-ready does not advance the head. The instruction sits in the ROB ready-but-uncommitted until the head reaches it.

Operation 3: Commit (in-order retirement)

At each cycle, look at the head:

- If `valid[head]` and `ready[head]`: commit. Free the `old_phys_dest[head]` back to the rename unit's free list. Advance head by 1. Clear `valid[head]`.
- Then look at `head + 1`: if valid and ready, commit it too. Advance head by 2.
- If neither: head doesn't move this cycle.

This is the 2-wide commit logic. Two ports for slot-A-commit and slot-B-commit, each emitting a signal to the rename unit (`commit_valid`, `commit_old_phys`).

2.5 The Count Update

At each cycle, count changes by:

$$\text{new count} = \text{count} + (\text{alloc_A} + \text{alloc_B}) - (\text{commit_A} + \text{commit_B})$$

This is a single arithmetic expression in Verilog — compute the deltas combinationaly, then update count synchronously.

2.6 Worked Example: A 4-Instruction Trace

ROB starts empty, head=0, tail=0, count=0. Free list has P32, P33, P34, P35 available.

Code:

```
I1: add  x5, x1, x2      # slow path (suppose it's slow for some reason)
I2: add  x6, x3, x4      # independent of I1
I3: add  x7, x6, x1      # depends on I2
I4: add  x8, x5, x7      # depends on I1 and I3
```

Cycle 1: Dispatch I1 (slot A), I2 (slot B).

- I1 → ROB[0]: {valid=1, ready=0, writes_rd=1, arch_dest=5, phys_dest=P32, old=P5}
- I2 → ROB[1]: {valid=1, ready=0, writes_rd=1, arch_dest=6, phys_dest=P33, old=P6}
- head=0, tail=2, count=2.
- I1 sits in an RS waiting for P1 and P2 (already ready). I1 is wakeup-ready and issues to ALU0. I2 also wakeup-ready, issues to ALU1.

Cycle 2: Dispatch I3 (slot A), I4 (slot B). I2 finishes; I1 doesn't yet.

Suppose I2's ALU computes in 1 cycle (since both PRAVAH ALUs are single-cycle, this is the common case — but for this example, imagine I1 needs 3 cycles for some reason).

- I3 → ROB[2]: {valid=1, ready=0, arch_dest=7, phys_dest=P34, old=P7}
- I4 → ROB[3]: {valid=1, ready=0, arch_dest=8, phys_dest=P35, old=P8}
- I3's RS source 1 (x6) gets P33 from rename map (because I2 just dispatched). It snoops PRF[33].ready — not yet set, since I2 just started executing.
- I4's RS source 1 (x5) gets P32; source 2 (x7) gets P34 (bypassed from I3 within this bundle).
- I2's ALU result is written to PRF[33], PRF[33].ready=1. ROB[1].ready=1.
- head=0, tail=4, count=4.

Cycle 3: I3 wakes up (P33 ready, P1 ready), issues. I1 still computing.

I3 sees PRF[33].ready=1, snoops match, issues to ALU0 (say). ALU0 begins computing I3.

Cycle 4: I3 finishes. I1 finishes (assuming a 3-cycle latency).

- ALU0 writes PRF[34], PRF[34].ready=1. ROB[2].ready=1.
- ALU1 (or wherever I1 ran) writes PRF[32], PRF[32].ready=1. ROB[0].ready=1.

Commit phase, cycle 4: Look at ROB[head=0]. ready=1. Commit I1. Free P5. head=1, count=3.

Then look at ROB[head=1]. ready=1 (from cycle 2). Commit I2. Free P6. head=2, count=2.

Both I1 and I2 committed in the same cycle, in original program order. This is the 2-wide commit at work.

Cycle 5: I4 wakes up (P32 ready, P34 ready), issues. ALU0 begins computing.

Cycle 6: I4 finishes. ROB[3].ready=1.

Commit phase, cycle 6: ROB[head=2]. I3 is ready. Commit. Free P7. head=3, count=1.

Then ROB[head=3]. I4 is ready. Commit. Free P8. head=4, count=0.

Pipeline is now empty. The architectural register file holds the final values of x5..x8.

Out-of-order timing observation: I3 finished (cycle 4) before I4 was even dispatched (also cycle 4, but slot B). But I3 still committed *after* I1 and I2 (which is correct: I3 is younger than them in program order). Out-of-order execution; in-order commit.

Concept Check 1

1. Why does the ROB use a separate **ready** bit per entry, rather than relying on the PRF's ready bit at **phys_dest**?
2. In the worked example, I2 was ready in cycle 2 but didn't commit until cycle 4. Why? What is being verified between those cycles?
3. Suppose the ROB is full (count=8) and ALU0 finishes an instruction whose ROB entry is at the head. Can a new dispatch happen this cycle?

3 Comprehensive Theory 2: The ALU and the CDB

[~30 min]

3.1 The ALU

The ALU is the simplest module in the design. It’s purely combinational (for our 1-cycle ALU).

Inputs:

- **op**: 4-bit operation code (from the RS’s stored op).
- **src1_val**, **src2_val**: 32-bit operand values (read from PRF at the corresponding phys reg indices).
- **valid**: 1 if this issue is real (not a no-op).
- Bookkeeping: **phys_dest**, **rob_idx** — pass-through, used by the writeback path.

Outputs:

- **result**: 32-bit ALU output.
- **result_valid**: 1 if computation is real.
- Bookkeeping pass-through: **phys_dest**, **rob_idx**.

Implementation: a giant **case** on the op code. $\text{ADD} = \text{src1} + \text{src2}$, $\text{SUB} = \text{src1} - \text{src2}$, $\text{AND} = \text{src1} \& \text{src2}$, etc. **ADDI** uses the immediate as **src2** (the dispatch logic places the immediate value into the RS’s **src2** slot as a special encoding — see Section 4 for how).

In PRAVAH the ALU is single-cycle: result is available the same cycle as inputs. In a real processor with higher clock speed, the ALU would be pipelined; but for our slow clock and small designs, single-cycle is fine.

3.2 The CDB Concept — and Why We Don’t Need a Real CDB

Classical Tomasulo’s Common Data Bus broadcast (tag, value) pairs to all reservation stations. Every RS had tag comparators on its operand-tag fields, capturing the value when the broadcasted tag matched.

In the modern PRF design, the values live in the PRF, not in RSs. So we don’t broadcast values. Instead, when an FU finishes, it writes the value to the PRF directly (writing to `PRF[phys_dest] = result`). Setting the ready bit there is enough — all RSs already know which phys reg they’re waiting for, and they snoop the PRF ready bits combinatorially.

So what is “the CDB” in PRAVAH? It’s the **arbitration logic** that picks which FU gets to write the PRF this cycle.

3.3 Why Arbitration Is Needed

The PRF has 2 write ports (Week 3 design). PRAVAH has 2 ALUs and (in Week 7) potentially MUL and LSU. If 3 FUs finish in the same cycle, only 2 can write the PRF this cycle. The other has to wait or be dropped.

Real processors handle this with stall signals to the FUs — the FU holds its output one more cycle if it can't write back. For PRAVAH, since we have only 2 FUs in the Week 6 scope (2 ALUs) and 2 PRF write ports, no contention can occur. Every ALU writeback gets a port. Simple.

But you should still structure your top-level as if arbitration could happen, because:

- Week 7's MUL adds a third potential writer.
- The CDB-arbitration habit is the right pattern for real designs.

3.4 The Writeback Connection: ALU $\rightarrow$ PRF and ALU $\rightarrow$ ROB

When an ALU finishes in cycle T, it must do three things simultaneously:

1. Write `result` to `PRF[phys_dest]`.
2. Set `PRF[phys_dest].ready = 1`.
3. Signal `ROB[rob_idx].ready = 1`.

All three happen at the same clock edge. In the top module, we wire ALU output ports to PRF write ports and to ROB mark-ready ports.

For PRAVAH's 2-ALU setup:

- ALU0's writeback goes to PRF write port 1 and ROB mark-ready port 1.
- ALU1's writeback goes to PRF write port 2 and ROB mark-ready port 2.

No arbitration needed — there's a 1-to-1 mapping. (In Week 7 if you add MUL, you'd need a mux at the PRF write ports.)

3.5 The Wakeup-Select-Issue Loop in Detail

This is the most performance-critical path in any OoO processor. Let's trace one full cycle:

Cycle T: ALU0 has finished computing instruction X. ALU0's writeback drives:

- PRF write port 1: `{en=1, addr=X.phys_dest, data=X.result}`.
- ROB mark-ready port 1: `{en=1, rob_idx=X.rob_idx}`.

These are *combinational* signals. The PRF and ROB don't see them until the clock edge at end of cycle T.

End of cycle T (clock edge):

- PRF latches: $\text{regs}[\text{X.phys_dest}] \leftarrow \text{X.result}$; $\text{ready}[\text{X.phys_dest}] \leftarrow 1$.
- ROB latches: $\text{ready}[\text{X.rob_idx}] \leftarrow 1$.

Start of cycle T+1: every RS combinationally snoops the PRF ready bits.

- RSs that were waiting on X.phys_dest now see ready=1 on that source.
- If both sources are ready (and busy=1), the RS raises wakeup_ready.
- Priority encoder picks one (or two, for 2-ALU) wakeup-ready RSs.
- The selected RSs drive their op, phys_src1, phys_src2, phys_dest, rob_idx to the corresponding ALUs.
- The ALUs read PRF[phys_src1] and PRF[phys_src2] (combinationally) and begin computing.

End of cycle T+1: the ALUs' results are stable at their output ports. If a single-cycle ALU, the writeback happens at this edge, repeating the loop.

One-cycle producer-to-consumer latency. An instruction that finishes in cycle T can have its consumer fire in cycle T+1. This is the design's IPC sweet spot.

3.6 The Bypass Tradeoff (Why We Don't Implement It)

A more aggressive design would let the consumer fire in the *same cycle* as the producer. This is called “early wakeup” or “operand forwarding” — the consumer's wakeup logic checks not just PRF.ready but also “is any FU writing this phys reg this cycle?” If yes, treat it as ready.

This shortens the producer-to-consumer latency to zero, but requires the wakeup logic, the priority encoder, and the new ALU launch all to fit in one clock cycle. For a slow clock (PRAVAH's target Fmax is ~50 MHz), this could be done, but the timing margin is tight. We don't implement it for Week 6.

For your Quartus timing report (Week 7), one of the things you'll examine is exactly this path. If it's the critical path, you know your design is limited by the wakeup-select loop.

Concept Check 2

1. Why is the producer-to-consumer latency in PRAVAH's design 1 cycle and not 0?
2. If two ALUs both finish in the same cycle and both target the same physical register (somehow), what happens to the PRF? (Hint: this shouldn't happen — but why?)
3. In the writeback path, ALU0 writes both the PRF and signals the ROB. What would happen if one of these writes was lost (e.g., PRF write fires but ROB mark-ready doesn't)?

4 Comprehensive Theory 3: Widened Wakeup-Select for 2-ALU Issue [~25 min]

In Week 4 we built a single-issue RS array (one issue per cycle, to one ALU). Now we extend to two simultaneous issues per cycle, one to each of two ALUs.

4.1 The Naive Approach (and Why It Doesn't Work)

Naive: “Pick the two lowest-index wakeup-ready RSs and issue them.”

Why this fails: the priority encoder logic is *not* composable. A flat “find lowest-index ready bit” encoder produces only one result. We need two distinct lowest-index ready bits.

4.2 The Mask-and-Encode Pattern

The standard pattern for picking N items from a one-hot vector is:

1. **Pick the first:** use a lowest-index priority encoder to find the lowest-index 1 in the vector. Call this `select_0`.
2. **Mask it out:** compute `masked_vector = original_vector & ~select_0`.
3. **Pick the second:** apply the same priority encoder to the masked vector. Call this `select_1`.
4. Continue if more selections needed.

For 2-ALU PRAVAH, we need 2 picks: `issue_select_0` and `issue_select_1`. Each is a one-hot vector indicating which RS issues to ALU0 or ALU1 respectively.

4.3 Concrete Verilog for 4-Entry RS with 2 Issues

```
1 // Inputs: rs_wakeup_ready[3:0] (computed as in Week 4)
2
3 // First pick: lowest-index wakeup_ready
4 wire [3:0] issue_select_0;
5 assign issue_select_0[0] = rs_wakeup_ready[0];
6 assign issue_select_0[1] = rs_wakeup_ready[1] & ~
    rs_wakeup_ready[0];
7 assign issue_select_0[2] = rs_wakeup_ready[2] & ~
    rs_wakeup_ready[1]
8                                     & ~
    rs_wakeup_ready
    [0];
9 assign issue_select_0[3] = rs_wakeup_ready[3] & ~
    rs_wakeup_ready[2]
```

```

10                                     & ~
                                     rs_wakeup_ready
                                     [1]
11                                     & ~
                                     rs_wakeup_ready
                                     [0];
12
13 // Mask out the first selection
14 wire [3:0] remaining = rs_wakeup_ready & ~issue_select_0;
15
16 // Second pick: lowest-index of remaining
17 wire [3:0] issue_select_1;
18 assign issue_select_1[0] = remaining[0];
19 assign issue_select_1[1] = remaining[1] & ~remaining[0];
20 assign issue_select_1[2] = remaining[2] & ~remaining[1] & ~
    remaining[0];
21 assign issue_select_1[3] = remaining[3] & ~remaining[2] & ~
    remaining[1]
22                                     & ~remaining[0];

```

4.4 Two Issue Buses

Each select drives a separate issue bus. The widened RS module has two complete sets of issue outputs — one for ALU0 (driven by select_0) and one for ALU1 (driven by select_1).

```

1 // ALU0 issue bus (driven by issue_select_0)
2 wire issue_0_valid = |issue_select_0;
3 wire [PHYS_W-1:0] issue_0_pj = (issue_select_0[0]) ? rs_pj[0]
4                               : (issue_select_0[1]) ? rs_pj[1]
5                               : (issue_select_0[2]) ? rs_pj[2]
6                               : rs_pj
7                               [3];
8
9 // ... similar muxes for op, pk, pd, rob_idx ...
10
11 // ALU1 issue bus (driven by issue_select_1) -- same pattern
    with select_1

```

4.5 Updating RS State

On a clock edge, both selected RSs clear their busy bits:

```

1 always @(posedge clk_i) begin
2     if (reset_i) begin
3         // ... reset all RSs ...
4     end else begin

```

```

5         integer i;
6         for (i = 0; i < NUM_RS; i = i + 1) begin
7             if (issue_select_0[i] | issue_select_1[i])
8                 rs_busy[i] <= 1'b0;
9         end
10        // ... dispatch updates ...
11    end
12end

```

The combinational logic guarantees `issue_select_0 & issue_select_1 == 0` (the mask ensures distinctness), so this OR doesn't double-write the same RS.

4.6 What About Multi-Cycle FUs (Like MUL)?

For Week 6 scope (ALU-only), we don't worry about this. But for Week 7's stretch goals:

- A pipelined MUL with throughput 1 (new MUL each cycle) doesn't need special handling — treat it like an ALU.
- A non-pipelined MUL (start-finish cycles) requires the MUL RS to hold the instruction even after issue, until the MUL is done. This needs a “free this RS slot when MUL completes” signal, not on issue.

4.7 Same-Cycle Wakeup From Just-Finished FU

Here's a subtlety. Suppose ALU0 finishes in cycle T, writing to PRF[X]. In cycle T+1, an RS that was waiting on X becomes wakeup-ready and can issue.

But can we go faster? What if in cycle T (same cycle as the writeback), the wakeup logic uses the *about-to-be-written* ready bit (taking it from the ALU's output port, not from the latched PRF)?

This is what real processors do with “early wakeup” or “operand forwarding from FU output.” It saves one cycle per RAW dependency.

PRAVAH does **not** do this. We rely on the latched PRF state, accepting a 1-cycle producer-to-consumer latency. Reasons:

- Simpler control logic.
- Easier to reason about correctness.
- Our test programs are not dependency-heavy enough to make this matter.

If you want to add it as a stretch goal in Week 7, the change is:

```

1 // Instead of:
2 wire src_ready = prf_ready[rs_pj[g]];
3
4 // Use:

```

```

5 wire src_ready = prf_ready[rs_pj[g]] |
6     (alu0_writeback_en & (alu0_writeback_phys ==
7     rs_pj[g])) |
    (alu1_writeback_en & (alu1_writeback_phys ==
    rs_pj[g]));

```

This is genuinely tricky — the priority encoder now must avoid selecting an RS whose source comes from a FU that’s about to write back, but only if that FU hasn’t actually finished yet. Easy to introduce bugs. Skip it unless you’re well ahead of schedule.

Concept Check 3

1. In the mask-and-encode pattern, why must we compute `issue_select_0` *before* computing `issue_select_1`, rather than in parallel?
2. Suppose only one RS is wakeup-ready. What is `issue_select_0`? `issue_select_1`? What happens at ALU0 and ALU1?
3. Why does PRAVAH leave the “early wakeup” optimization out?

5 Comprehensive Theory 4: The Top-Level Integration [~20 min]

This section describes how all the modules wire together. `pravah_top.v` instantiates everything and connects the signals. The wiring is the hard part of Week 6.

5.1 The Modules in PRAVAH_TOP

- `fetch`: drives instruction memory; produces 2-wide instructions to decode.
- `decode × 2`: parallel decoders for slot A and slot B.
- `rename_unit`: 2-wide rename (built in Week 5).
- `dispatch`: combinational wiring (built in Week 5).
- `reservation_station`: 4-entry, 2-dispatch, 2-issue (widened from Week 4 & 5).
- `register_file`: PRF (built in Week 3).
- `alu × 2`: two ALUs (built this week).
- `rob`: 8-entry, 2-wide dispatch, 2-wide commit (built this week).

5.2 The Data Flow Diagram

Front-end: <code>fetch</code> → <code>decode×2</code> → <code>rename</code> → <code>dispatch</code>
Back-end (parallel): <code>dispatch</code> → <code>rs</code> (write) and <code>rob</code> (alloc) <code>rs</code> (issue) → <code>alu</code> (compute) <code>alu</code> (writeback) → <code>prf</code> (write) and <code>rob</code> (mark ready)
Commit: <code>rob</code> (commit) → <code>rename</code> (free phys reg)
Snooping: <code>rs</code> (snoop) → <code>prf</code> (ready bits) and back

5.3 The Connection Table

Every output of every module connects to inputs of other modules. The clearest way to describe this is a connection table:

Driver (output)	Consumer (input)	What
fetch.pc_A_o, instr_A_o, valid_A_o	decode_A	Slot A instruction
fetch.pc_B_o, instr_B_o, valid_B_o	decode_B	Slot B instruction
decode_A.*	rename, dispatch	Slot A decoded fi
decode_B.*	rename, dispatch	Slot B decoded fi
rename.*_phys_o	dispatch	Physical regs for
dispatch.rs_disp*_o	reservation_station	RS dispatch ports
dispatch.rob_alloc*_o	rob	ROB dispatch po
rob.tail_o, full_o	dispatch	Back-pressure
rs.snoop*_addr_o	prf	RS reading PRF
prf.rd_ready_o	rs.snoop*_ready_i	PRF ready bits to
rs.issue0*_o	alu0	Issue to ALU0
rs.issue1*_o	alu1	Issue to ALU1
alu.src1_addr_o	prf.rd_addr*	ALU reads PRF
prf.rd_data*	alu.src*_val_i	PRF to ALU
alu0.result_o, valid_o, phys_dest_o	prf (write port 1), rob (mark-ready 1)	ALU0 writeback
alu1.result_o, valid_o, phys_dest_o	prf (write port 2), rob (mark-ready 2)	ALU1 writeback
rob.commit*_valid_o, commit*_old_phys_o	rename (commit port)	Commit feedback
dispatch.fe_stall_o	fetch	Back-pressure

This table is your master reference for the top module. Print it.

5.4 Allocate-Ports: Clear PRF Ready on Dispatch

A detail easy to miss: when an instruction is dispatched and gets a fresh phys reg from the rename unit, the PRF's ready bit at that phys reg must be *cleared* (since the value isn't computed yet). The PRF has an `alloc_en_i` / `alloc_addr_i` port for exactly this purpose (Week 3 design).

In the top module:

```

1 // PRF allocate ports driven by dispatch's new phys_dest (when
  writes_rd)
2 assign prf_alloc_en_1    = rob_alloc_A & rob_A_writes_rd;
3 assign prf_alloc_addr_1 = rob_A_phys_dest;
4 assign prf_alloc_en_2    = rob_alloc_B & rob_B_writes_rd;
5 assign prf_alloc_addr_2 = rob_B_phys_dest;

```

If you forget this, the PRF still says “ready=1” for the freshly allocated phys reg, and consumers (RSs) immediately fire on it, reading garbage. **This is the most common Week 6 silent failure.**

The Week 3 PRF design has only one alloc port; for 2-wide we need two. Widen the PRF accordingly (add a second `alloc_en` / `alloc_addr` pair).

5.5 Immediate Operands

ADDI is an issue. Its second operand is an immediate, not a register value. How does the ALU get it?

Two options:

- **Carry imm through the RS:** the RS stores an additional “imm” field; the ALU has both `src2_val` (from PRF) and `imm`; the op decoder picks which to use.
- **Use a special phys reg for the immediate:** dispatch “injects” the immediate value into a reserved PRF entry and uses its phys reg as `src2`. (Complex, error-prone.)

PRAVAH uses option 1. The RS gets an extra `imm` field (32 bits) and a 1-bit `use_imm` flag. When `use_imm` is set, the ALU uses the `imm` field instead of reading `src2` from PRF. The dispatch logic sets these fields when it sees an immediate-using opcode.

This adds modest complexity to the RS and ALU but is much cleaner than the alternatives.

Concept Check 4

1. Why do we need to clear the PRF ready bit when a phys reg is allocated, not when the instruction starts executing?
2. In the connection table, the RS snoops the PRF ready bits combinatorially. Why does this work without a clock edge between the PRF update and the RS sees it?
3. If you forget to wire the `dispatch.fe_stall_o` signal to `fetch's fe_stall_i`, what happens?

6 Verilog Implementations [~1 hour reading]

6.1 Module: rob.v

```
1 module rob #(
2     parameter ROB_SIZE      = 8,
3     parameter PHYS_ADDR_W   = 6,
4     parameter ARCH_ADDR_W   = 5,
5     parameter ROB_ADDR_W    = 3    // log2(8)
6 ) (
7     input  wire              clk_i,
8     input  wire              reset_i,
9
10    // Dispatch ports (slot A and slot B)
11    input  wire               alloc_A_i,
12    input  wire               writes_rd_A_i,
13    input  wire [ARCH_ADDR_W-1:0] arch_dest_A_i,
14    input  wire [PHYS_ADDR_W-1:0] phys_dest_A_i,
15    input  wire [PHYS_ADDR_W-1:0] old_phys_dest_A_i,
16
17    input  wire               alloc_B_i,
18    input  wire               writes_rd_B_i,
19    input  wire [ARCH_ADDR_W-1:0] arch_dest_B_i,
20    input  wire [PHYS_ADDR_W-1:0] phys_dest_B_i,
21    input  wire [PHYS_ADDR_W-1:0] old_phys_dest_B_i,
22
23    // Outputs for dispatch
24    output wire [ROB_ADDR_W-1:0] tail_o,
25    output wire                  full_o,
26
27    // Mark-ready ports (from FUs)
28    input  wire                mark_ready_0_en_i,
29    input  wire [ROB_ADDR_W-1:0] mark_ready_0_idx_i,
30    input  wire                mark_ready_1_en_i,
31    input  wire [ROB_ADDR_W-1:0] mark_ready_1_idx_i,
32
33    // Commit ports (to rename unit)
34    output wire                commit_A_valid_o,
35    output wire [PHYS_ADDR_W-1:0] commit_A_old_phys_o,
36    output wire                commit_B_valid_o,
37    output wire [PHYS_ADDR_W-1:0] commit_B_old_phys_o
38 );
39
40    // ROB entry storage (decomposed into parallel arrays for
41    clarity)
```

```

41     reg                                rob_valid      [0:ROB_SIZE-1];
42     reg                                rob_ready       [0:ROB_SIZE-1];
43     reg                                rob_writes_rd   [0:ROB_SIZE-1];
44     reg [ARCH_ADDR_W-1:0]              rob_arch_dest  [0:ROB_SIZE-1];
45     reg [PHYS_ADDR_W-1:0]              rob_phys_dest  [0:ROB_SIZE-1];
46     reg [PHYS_ADDR_W-1:0]              rob_old_phys   [0:ROB_SIZE-1];
47
48     // Pointers and count
49     reg [ROB_ADDR_W-1:0]                head, tail;
50     reg [ROB_ADDR_W:0]                  count;        // extra bit to represent
51                                                    ROB_SIZE
52
53     // ----- Combinational outputs -----
54     assign tail_o = tail;
55     assign full_o = (count == ROB_SIZE);
56
57     // ----- Commit logic (combinational selection) -----
58     // Slot A: head entry committable iff valid and ready
59     wire head_committable = rob_valid[head] & rob_ready[head];
60     wire head1_idx        = (head + 1) % ROB_SIZE;
61     wire head1_committable = rob_valid[head1_idx] & rob_ready[
62         head1_idx]
63         & head_committable;
64
65     assign commit_A_valid_o      = head_committable;
66     assign commit_A_old_phys_o   = rob_old_phys[head];
67     assign commit_B_valid_o      = head1_committable;
68     assign commit_B_old_phys_o   = rob_old_phys[head1_idx];
69
70     // ----- Helpers for state updates -----
71     wire do_alloc_A  = alloc_A_i;
72     wire do_alloc_B  = alloc_B_i;
73     wire do_commit_A = head_committable;
74     wire do_commit_B = head1_committable;
75
76     wire [1:0] num_allocs  = do_alloc_A + do_alloc_B;
77     wire [1:0] num_commits = do_commit_A + do_commit_B;
78
79     // ----- Synchronous updates -----
80     integer i;
81     always @(posedge clk_i) begin
82         if (reset_i) begin
83             for (i = 0; i < ROB_SIZE; i = i + 1) begin
84                 rob_valid[i] <= 1'b0;
85                 rob_ready[i] <= 1'b0;

```

```

84     end
85     head  <= 0;
86     tail  <= 0;
87     count <= 0;
88 end else begin
89
90     // Allocate at tail (slot A)
91     if (do_alloc_A) begin
92         rob_valid    [tail] <= 1'b1;
93         rob_ready    [tail] <= 1'b0;
94         rob_writes_rd[tail] <= writes_rd_A_i;
95         rob_arch_dest[tail] <= arch_dest_A_i;
96         rob_phys_dest[tail] <= phys_dest_A_i;
97         rob_old_phys [tail] <= old_phys_dest_A_i;
98     end
99
100    // Allocate at tail+1 (slot B), only if slot A also
101    allocated
102    if (do_alloc_A & do_alloc_B) begin
103        rob_valid    [(tail + 1) % ROB_SIZE] <= 1'b1;
104        rob_ready    [(tail + 1) % ROB_SIZE] <= 1'b0;
105        rob_writes_rd[(tail + 1) % ROB_SIZE] <=
106            writes_rd_B_i;
107        rob_arch_dest[(tail + 1) % ROB_SIZE] <=
108            arch_dest_B_i;
109        rob_phys_dest[(tail + 1) % ROB_SIZE] <=
110            phys_dest_B_i;
111        rob_old_phys [(tail + 1) % ROB_SIZE] <=
112            old_phys_dest_B_i;
113    end
114
115    // Mark-ready from FUs
116    if (mark_ready_0_en_i)
117        rob_ready[mark_ready_0_idx_i] <= 1'b1;
118    if (mark_ready_1_en_i)
119        rob_ready[mark_ready_1_idx_i] <= 1'b1;
120
121    // Commit: invalidate head entries
122    if (do_commit_A)
123        rob_valid[head] <= 1'b0;
124    if (do_commit_B)
125        rob_valid[(head + 1) % ROB_SIZE] <= 1'b0;
126
127    // Advance pointers
128    tail <= (tail + num_allocs) % ROB_SIZE;

```

```

124         head <= (head + num_commits) % ROB_SIZE;
125         count <= count + num_allocs - num_commits;
126     end
127 end
128
129 endmodule

```

Subtle point on simultaneous mark-ready and commit: If a FU marks ROB[K] ready in the same cycle that commit is examining ROB[K], does the commit see ready=1? Answer: **no**, because the mark-ready writes to `rob_ready[K]` happen at the clock edge, while commit reads the value at the start of the cycle. The committable signal uses the old value of `rob_ready[K]`. The instruction commits the next cycle.

This adds a 1-cycle latency between FU completion and commit. For PRAVAH's small ROB this is fine; for larger machines it can be optimized with bypass, but we accept the latency.

6.2 Module: alu.v

```

1 module alu #(
2     parameter OP_WIDTH      = 4,
3     parameter DATA_WIDTH   = 32,
4     parameter PHYS_ADDR_W   = 6,
5     parameter ROB_ADDR_W    = 3
6 ) (
7     input wire               clk_i,    // unused; ALU is
8     combinational
9     input wire               reset_i,  // unused
10
11     // Issue inputs (from RS)
12     input wire               valid_i,
13     input wire [OP_WIDTH-1:0] op_i,
14     input wire [DATA_WIDTH-1:0] src1_val_i,
15     input wire [DATA_WIDTH-1:0] src2_val_i,
16     input wire [DATA_WIDTH-1:0] imm_i,
17     input wire               use_imm_i,
18     input wire [PHYS_ADDR_W-1:0] phys_dest_i,
19     input wire [ROB_ADDR_W-1:0] rob_idx_i,
20
21     // Writeback outputs
22     output wire               result_valid_o,
23     output wire [DATA_WIDTH-1:0] result_o,
24     output wire [PHYS_ADDR_W-1:0] phys_dest_o,
25     output wire [ROB_ADDR_W-1:0] rob_idx_o
26 );

```

```

26
27 wire [DATA_WIDTH-1:0] operand2 = use_imm_i ? imm_i :
    src2_val_i;
28 reg [DATA_WIDTH-1:0] result_internal;
29
30 always @* begin
31     case (op_i)
32         4'd0:    result_internal = src1_val_i + operand2;
33                 // ADD/ADDI
34         4'd1:    result_internal = src1_val_i - operand2;
35                 // SUB
36         4'd2:    result_internal = src1_val_i & operand2;
37                 // AND
38         4'd3:    result_internal = src1_val_i | operand2;
39                 // OR
40         4'd4:    result_internal = src1_val_i ^ operand2;
41                 // XOR
42         4'd5:    result_internal = src1_val_i << operand2
43                 [4:0]; // SLL
44         4'd6:    result_internal = src1_val_i >> operand2
45                 [4:0]; // SRL
46         4'd7:    result_internal = ($signed(src1_val_i) <
47                 $signed(operand2)) ? 32'd1 : 32'd0; // SLT
48         4'd8:    result_internal = src1_val_i + imm_i;
49                 // ADDI explicitly
50         default: result_internal = 32'b0;
51     endcase
52 end
53
54 // Pass-through outputs
55 assign result_valid_o = valid_i;
56 assign result_o       = result_internal;
57 assign phys_dest_o    = phys_dest_i;
58 assign rob_idx_o      = rob_idx_i;
59
60 endmodule

```

Note: The ALU is fully combinational. It has clock and reset ports declared for compatibility with later changes (e.g., if you make it pipelined in Week 7), but they're unused.

6.3 Module: Top-Level Skeleton

The top module is mostly wiring. Here's a skeleton showing the instantiation structure:

```

1 module pravah_top (

```

```

2   input wire clk_i,
3   input wire reset_i
4   // (no external interface for now; everything internal)
5 );
6
7   // ----- All inter-module wires -----
8   // Fetch outputs
9   wire [31:0] fetch_pc_A, fetch_pc_B;
10  wire [31:0] fetch_instr_A, fetch_instr_B;
11  wire        fetch_valid_A, fetch_valid_B;
12
13  // Decode outputs
14  wire [3:0]   dec_A_op, dec_B_op;
15  wire [4:0]   dec_A_rs1_arch, dec_A_rs2_arch, dec_A_rd_arch;
16  wire [4:0]   dec_B_rs1_arch, dec_B_rs2_arch, dec_B_rd_arch;
17  wire        dec_A_writes_rd, dec_B_writes_rd;
18  wire [31:0] dec_A_imm, dec_B_imm;
19  wire [1:0]   dec_A_fu_type, dec_B_fu_type;
20  wire        dec_A_valid, dec_B_valid;
21
22  // Rename outputs
23  wire [5:0]   rn_A_rs1_phys, rn_A_rs2_phys, rn_A_rd_phys,
24              rn_A_rd_old_phys;
25  wire [5:0]   rn_B_rs1_phys, rn_B_rs2_phys, rn_B_rd_phys,
26              rn_B_rd_old_phys;
27  wire        rn_stall_A, rn_stall_B;
28
29  // Dispatch outputs
30  wire        disp_fe_stall;
31  wire        rob_alloc_A, rob_alloc_B;
32  wire [4:0]   rob_arch_dest_A, rob_arch_dest_B;
33  wire [5:0]   rob_phys_dest_A, rob_phys_dest_B;
34  wire [5:0]   rob_old_phys_A, rob_old_phys_B;
35  wire        rob_writes_rd_A, rob_writes_rd_B;
36  wire        rs_disp_A_valid, rs_disp_B_valid;
37  // ... (more dispatch outputs)
38
39  // ROB outputs
40  wire [2:0]   rob_tail;
41  wire        rob_full;
42  wire        rob_commit_A_valid, rob_commit_B_valid;
43  wire [5:0]   rob_commit_A_old_phys, rob_commit_B_old_phys;
44
45  // RS outputs (issue buses)
46  wire        rs_issue_0_valid, rs_issue_1_valid;

```

```

45  wire [3:0]  rs_issue_0_op, rs_issue_1_op;
46  wire [5:0]  rs_issue_0_pj, rs_issue_1_pj, rs_issue_0_pk,
      rs_issue_1_pk;
47  wire [5:0]  rs_issue_0_pd, rs_issue_1_pd;
48  wire [2:0]  rs_issue_0_rob_idx, rs_issue_1_rob_idx;
49  // ... (snoop wires)
50
51  // ALU outputs
52  wire        alu0_result_valid, alu1_result_valid;
53  wire [31:0] alu0_result, alu1_result;
54  wire [5:0]  alu0_phys_dest, alu1_phys_dest;
55  wire [2:0]  alu0_rob_idx, alu1_rob_idx;
56
57  // PRF outputs
58  wire [31:0] prf_rd_data_1, prf_rd_data_2, prf_rd_data_3,
      prf_rd_data_4;
59  wire        prf_rd_ready_1, prf_rd_ready_2, prf_rd_ready_3,
      prf_rd_ready_4;
60  // ... (more PRF wires)
61
62  // ----- Module instances -----
63  fetch u_fetch (...);
64  decode u_decode_A (.instr_i(fetch_instr_A), .valid_i(
      fetch_valid_A), ...);
65  decode u_decode_B (.instr_i(fetch_instr_B), .valid_i(
      fetch_valid_B), ...);
66  rename_unit u_rename (...);
67  dispatch u_dispatch (...);
68  reservation_station u_rs (...);
69  register_file u_prf (...);
70  alu u_alu0 (...);
71  alu u_alu1 (...);
72  rob u_rob (...);
73
74  // ----- Wiring connections -----
75  // PRF read ports: 2 for ALU0 (src1, src2), 2 for ALU1 (src1,
      src2)
76  assign prf_rd_addr_1 = rs_issue_0_pj;
77  assign prf_rd_addr_2 = rs_issue_0_pk;
78  assign prf_rd_addr_3 = rs_issue_1_pj;
79  assign prf_rd_addr_4 = rs_issue_1_pk;
80
81  // ALU0 reads PRF
82  assign alu0_src1_val = prf_rd_data_1;
83  assign alu0_src2_val = prf_rd_data_2;

```

```

84
85 // ALU1 reads PRF
86 assign alu1_src1_val = prf_rd_data_3;
87 assign alu1_src2_val = prf_rd_data_4;
88
89 // PRF write ports: ALU0 to port 1, ALU1 to port 2
90 assign prf_wr_en_1    = alu0_result_valid;
91 assign prf_wr_addr_1 = alu0_phys_dest;
92 assign prf_wr_data_1 = alu0_result;
93
94 assign prf_wr_en_2    = alu1_result_valid;
95 assign prf_wr_addr_2 = alu1_phys_dest;
96 assign prf_wr_data_2 = alu1_result;
97
98 // PRF allocate ports from dispatch
99 assign prf_alloc_en_1    = rob_alloc_A & rob_writes_rd_A;
100 assign prf_alloc_addr_1 = rob_phys_dest_A;
101 assign prf_alloc_en_2    = rob_alloc_B & rob_writes_rd_B;
102 assign prf_alloc_addr_2 = rob_phys_dest_B;
103
104 // ROB mark-ready from ALUs
105 assign rob_mark_ready_0_en = alu0_result_valid;
106 assign rob_mark_ready_0_idx = alu0_rob_idx;
107 assign rob_mark_ready_1_en = alu1_result_valid;
108 assign rob_mark_ready_1_idx = alu1_rob_idx;
109
110 // ROB commit to rename unit's commit port
111 // (rename unit has only one commit port; if both A and B
112 //    commit,
113 // we'll need to either widen rename or accept a 1-cycle
114 //    delay)
115 // For Week 6: serialize. Slot A commits this cycle; slot B
116 //    next cycle if both ready.
117 assign rename_commit_valid    = rob_commit_A_valid;
118 assign rename_commit_old_phys = rob_commit_A_old_phys;
119 // (TODO Week 7: widen rename to 2-commit per cycle)
120
121 endmodule

```

Important caveat on the rename unit's commit port: The Week 5 rename unit has a single commit port. If the ROB wants to commit 2 instructions per cycle, only one's free can be returned per cycle. For Week 6's small test programs, this isn't a bottleneck (we rarely commit 2 in a cycle), so we accept it. Week 7 polishing should widen the rename unit's commit port to 2.

6.4 Widening the PRF for 2 Allocate Ports

The Week 3 PRF has one allocate port. For PRAVAH 2-wide, we need two. Modify the PRF:

```
1 // Add to the module's port list
2 input wire alloc_en1_i,
3 input wire [ADDR_WIDTH-1:0] alloc_addr1_i,
4 input wire alloc_en2_i,
5 input wire [ADDR_WIDTH-1:0] alloc_addr2_i,
6
7 // In the synchronous block, add a second clear:
8 if (alloc_en2_i &&
9     !(wr_en1_i && (wr_addr1_i == alloc_addr2_i)) &&
10     !(wr_en2_i && (wr_addr2_i == alloc_addr2_i))) begin
11     ready[alloc_addr2_i] <= 1'b0;
12 end
```

Edge case: What if alloc1 and alloc2 target the same address? They shouldn't (rename unit allocates distinct phys regs), but defensive code: only one needs to clear the ready bit.

Concept Check 5

1. In the ROB, why is the simultaneous mark-ready and commit case important to think through?
2. The ALU has clock and reset ports but doesn't use them. Why declare them anyway?
3. In pravah_top, the rename unit's single commit port is connected to slot A only. What might go wrong if slot B also wants to commit in the same cycle?

7 The Week 6 Bug-Hunt Checklist [PRINT THIS]

When your simulation produces wrong results, work through this list **in order**. Each item targets a specific failure mode I’ve seen many times in real Week 6 submissions.

7.1 Failure 1: “Nothing happens at all.”

Symptom: reset releases, but no instruction ever dispatches or commits.

Diagnose:

- Check `fetch.pc` — is it advancing? If not, `fe_stall_i` might be stuck high. Trace it back to dispatch.
- Check `decode_A.valid_o` — if 0, fetch’s valid bits aren’t being set. Check the imem initialization (`$readmemh` path).
- Check `rename.stall_A_o` — if 1, the free list is empty even at startup. The reset code didn’t initialize the free list correctly.

7.2 Failure 2: “First instruction dispatches but nothing executes.”

Symptom: ROB allocates entries, but the ALU never fires; `alu0.result_valid_o` stays 0.

Diagnose:

- Check the PRF allocate ports: are they being driven on dispatch? If `prf_alloc_en_1 == 0`, the freshly-allocated phys reg still has `ready=1` from reset, so the RS instantly wakes up and... wait, that should mean issue happens.
- Check `rs.snoop_pj_ready[0]` for the first RS — is it 1? If 0, the source phys reg’s ready bit isn’t being set.
- Check the source phys reg’s ready in PRF directly: `prf.ready[0]` (if reading x0 which maps to P0). At reset, this should be 1.

Most common cause: you forgot to wire the `dispatch.rs_disp*_valid_o` signal to RS, so the RS never sees the dispatch. The instruction is in the ROB but not in any RS. Check the wire.

7.3 Failure 3: “Lost CDB broadcast” (PRF write or ROB mark-ready missed).

Symptom: ALU completes, but the dependent instruction never wakes up. Or ROB head sits at a valid entry forever waiting for `ready=1`.

Diagnose:

- Compare `alu0.result_valid_o` (the ALU’s writeback signal) to `prf.wr_en1_i` and `rob.mark_ready_0_en_i`. They should all match.
- If they don’t: a wiring error in the top module. Add an assertion in your testbench.

Most common cause: typo in the top module. `prf.wr_en_1` typed as something slightly different elsewhere.

7.4 Failure 4: “ROB deadlock” (full ROB; head doesn’t advance).

Symptom: ROB fills up; dispatch stalls; ROB head sits at an entry that’s not ready; no progress.

Diagnose:

- Check `rob.rob_ready[head]`: is it 1 or 0?
- If 0, the ALU never wrote back. Trace the issue chain: did the RS issue? Did the ALU receive valid inputs?
- Most likely: the head instruction’s source phys reg is permanently stuck not-ready. Check why.

Most common cause: the instruction’s source was a phys reg that was *allocated* (ready cleared) but the producing instruction was never issued (because its sources were also not-ready, recursively). This forms a dependency chain back to a phys reg that was allocated but never written.

The fix: verify that the chain of dependencies eventually grounds out at a phys reg that was ready at reset (i.e., one of the initial 0..31 mappings). If the chain goes around in a cycle, you have a bug in the rename unit’s bypass logic.

7.5 Failure 5: “Off-by-one in ROB pointer.”

Symptom: instructions are committed out of order, or with wrong `arch_dest` mappings, or the wrong phys reg is freed.

Diagnose:

- Print every ROB allocation: `(tail, arch_dest, phys_dest, old_phys_dest)`.
- Print every ROB commit: `(head, arch_dest, phys_dest, old_phys_dest)`.
- Compare: the sequence of commits must exactly match the sequence of allocations (FIFO order).

Most common cause: `(tail + 1) % ROB_SIZE` computed but `tail` updated as `tail + num_allocs` without modulo. Result: tail wraps incorrectly when it should overflow.

Fix: ensure every `tail` and `head` update has a `% ROB_SIZE` (or equivalent bit-masking).

7.6 Failure 6: “Stale rename map” (intra-bundle bypass not working).

Symptom: `add x5, x1, x2 / sub x6, x5, x3` dispatched in same cycle: the sub reads x5’s OLD value, not the add’s freshly-computed one.

Diagnose:

- Check the rename unit’s bypass mux: `B.src1_uses_A = A.needs_alloc & (A.rd_arch == B.src1_arch)`.
- Probe `disp_B.rs1_phys_o` in the simulator: it should equal A’s `disp_A.rd_phys_o`, NOT `rename_map[B.src1_arch]`.

Most common cause: the bypass condition logic was correct in Week 5 but lost during Week 6 integration. Re-read your `rename_unit.v` carefully.

7.7 Failure 7: “Free list corruption.”

Symptom: multiple architectural registers end up mapped to the same phys reg.

Diagnose:

- Print the rename map at every cycle. Look for two entries with the same value.
- Print every push and pop of the free list. They should be balanced: each phys reg is popped exactly once before being pushed back.

Most common cause: commit fires but the rename unit’s commit port wasn’t wired. Or: the free list pointer advances by the wrong amount.

7.8 The General Debugging Strategy

When you don’t know what’s wrong:

1. Add `$display` statements (in the testbench, not in RTL!) for every major event: dispatch, issue, complete, commit.
2. Run the smallest possible test program (3-4 instructions). Trace by hand what should happen vs. what does happen.
3. Look at the ModelSim waveform. Add wires recursively: `add wave -recursive *`.
4. For each “something didn’t happen” diagnosis, find the first cycle where the divergence starts. Work backward from there.

The first time everything works end-to-end is going to be a memorable moment. Persistence is what gets you there.

8 The Integration Test — Dot Product [~1 hour]

8.1 The Test Program

A 10-instruction dot product computes the sum of products of two short vectors. Since we don't have memory operations in Week 6 (those are stretch goals), we use a simplified version: compute sum of pre-loaded values.

```
# Test 2 -- dot-product-like (ALU only)
# Pre-load: x1=2, x2=3, x3=5, x4=7
addi x1, x0, 2
addi x2, x0, 3
addi x3, x0, 5
addi x4, x0, 7

# Compute partial products
add  x5, x1, x2      # x5 = 2 + 3 = 5 (we use add as a proxy for mul)
add  x6, x3, x4      # x6 = 5 + 7 = 12

# Sum the partial products
add  x7, x5, x6      # x7 = 5 + 12 = 17

# A final WAW + WAR for test coverage
add  x5, x7, x7      # x5 = 17 + 17 = 34 (WAW on x5)
add  x6, x5, x1      # x6 = 34 + 2 = 36
```

Final register state (which the testbench checks):

- $x1 = 2, x2 = 3, x3 = 5, x4 = 7$
- $x5 = 34$ (final WAW resolution)
- $x6 = 36$ (final write)
- $x7 = 17$

Encode to hex using the Python helper from Week 5 (extend it to the new instruction list).

8.2 The Testbench Structure

tb_pravah_top.v does:

```
1 module tb_pravah_top;
2
3     reg clk = 0, reset = 1;
4     always #5 clk = ~clk;
5
```

```

6      // Instantiate the top module
7      pravah_top dut (.clk_i(clk), .reset_i(reset));
8
9      // Load instruction memory from file
10     initial begin
11         $readmemh("dot_product.hex", dut.u_fetch.imem);
12         #20 reset = 0;
13         #2000 $finish;
14     end
15
16     // Display every commit
17     always @(posedge clk) begin
18         if (dut.u_rob.do_commit_A) begin
19             $display("Cycle %0d: COMMIT slot A: rob_idx=%0d, arch
20                     =%0d, phys=%0d, val=%0h",
21                     $time, dut.u_rob.head, dut.u_rob.
22                     rob_arch_dest[dut.u_rob.head],
23                     dut.u_rob.rob_phys_dest[dut.u_rob.head],
24                     dut.u_prf.regs[dut.u_rob.rob_phys_dest[dut.
25                     u_rob.head]]);
26         end
27         if (dut.u_rob.do_commit_B) begin
28             // similar for slot B
29         end
30     end
31
32     // At the end, verify the architectural register file by
33     // reading via rename map
34     initial begin
35         #1500;
36         $display("Final architectural state:");
37         $display(" x1 = %0d (expected 2)", dut.u_prf.regs[dut.
38             u_rename.rename_map[1]]);
39         $display(" x2 = %0d (expected 3)", dut.u_prf.regs[dut.
40             u_rename.rename_map[2]]);
41         $display(" x5 = %0d (expected 34)", dut.u_prf.regs[dut.
42             u_rename.rename_map[5]]);
43         $display(" x6 = %0d (expected 36)", dut.u_prf.regs[dut.
44             u_rename.rename_map[6]]);
45         $display(" x7 = %0d (expected 17)", dut.u_prf.regs[dut.
46             u_rename.rename_map[7]]);
47         $finish;
48     end
49
50 endmodule

```

Reading architectural state: since architectural register xk is currently stored at physical register `rename_map[k]`, we look up the mapping and then read the PRF. If the rename map says `rename_map[5] = P37`, then `x5`'s value is `prf.regs[37]`.

8.3 Running and Verifying

Now, you know how to do this

The expected output (abbreviated):

```
Cycle 40: COMMIT slot A: rob_idx=0, arch=1, phys=32, val=2
Cycle 40: COMMIT slot B: rob_idx=1, arch=2, phys=33, val=3
Cycle 50: COMMIT slot A: rob_idx=2, arch=3, phys=34, val=5
...
Final architectural state:
  x1 = 2 (expected 2)
  x2 = 3 (expected 3)
  x5 = 34 (expected 34)
  x6 = 36 (expected 36)
  x7 = 17 (expected 17)
PASS
```

If any of these don't match, you have work to do. Section 6's checklist is your tool.

8.4 Take Two Waveform Screenshots

For the deliverable:

Screenshot 1: OoO execution. A cycle where two instructions issue to ALU0 and ALU1 in parallel, and at least one of them is younger (later in program order) than something still waiting in an RS. This visually proves OoO is happening.

Screenshot 2: In-order commit. A cycle showing the ROB head advancing through instructions in their original program order, with the architectural register file updating correctly.

9 Week 6 Deliverables — Milestone 3

In Your Repo

- `rtl/rob.v` — 8-entry reorder buffer.
- `rtl/alu.v` — single-cycle ALU.
- `rtl/reservation_station.v` — widened to 2-issue (replaces Weeks 4-5 version).
- `rtl/register_file.v` — widened with 2 allocate ports (replaces Week 3 version).
- `rtl/pravah_top.v` — top-level integration.
- `tb/tb_pravah_top.v` — end-to-end testbench.
- `programs/dot_product.hex` — benchmark program.
- `sim/waveforms/ooo_execution.png` — screenshot 1.
- `sim/waveforms/in_order_commit.png` — screenshot 2.
- Updated `learning_log.md` — this is more important than ever; write what you learned from the integration debugging.

What Mentor Will Check (Milestone 3 Gate)

1. Test program runs to completion without deadlock.
2. Final register state matches expected values exactly.
3. OoO execution is visible in the waveform.
4. In-order commit is visible in the waveform.
5. Plain Verilog only.
6. All previous modules (Weeks 3–5) still pass their unit tests.

If You Don't Finish on Time

This is the hardest week. Realistically, only some mentees finish Milestone 3 on time. If you don't:

1. Reach out if you are stuck. Don't disappear.
2. Use Week 7's first few days to finish Milestone 3 before starting performance work.
3. Consider scope reduction: a single-issue version (1 ALU, dispatch one at a time) is still a passing project. The PRAVAH 2-wide is the target, but a working 1-wide OoO with ROB is far better than a half-broken 2-wide.

Time Estimate

- Theory + reading (Sections 1–4): ~2.5 hours.
- Implementing rob.v + testbench: ~2.5 hours.
- Implementing alu.v: ~30 min.
- Widening reservation_station.v: ~1.5 hours.
- Widening register_file.v (2 alloc ports): ~30 min.
- Writing pravah_top.v: ~1.5 hours.
- Integration debugging: ~2 hours (this is the realistic number; budget for it).

Total: 10–11 hours. You will likely go over. Plan for it.

A Appendix: Solutions to Concept Checks

Concept Check 1 (ROB)

1. Because the PRF.ready bit indicates “a value has been written to this phys reg.” But the PRF doesn’t know which ROB entry that phys reg corresponds to. The ROB needs its own ready bit, indexed by ROB position, to know which entries are ready to commit. The ROB.ready bit is set by the FU’s writeback signal, which carries the ROB index along with the result.
2. I2 was ready in cycle 2, but the ROB head is I1, which wasn’t ready until cycle 4. Between cycles 2 and 4, I2 sat in the ROB with ready=1, waiting. The ROB enforces commit-in-program-order, so I2 cannot commit until everything before it (I1) commits.
3. Yes! Dispatch and commit are independent. In a single cycle: ALU0 marks the head ROB entry ready; commit logic sees head valid and ready, commits it, advances head, frees phys reg back to free list. Now there’s space at the new tail for a new dispatch.

Concept Check 2 (ALU and CDB)

1. Because the producer (ALU) writes to the PRF at the clock edge ending cycle T. The PRF.ready bit becomes visible to the consumer (RS) at the start of cycle T+1. The RS computes wakeup combinationaly in T+1 and issues in T+1 (which actually executes in T+1 since the ALU is combinational). So consumer fires in T+1, producer wrote in T — 1-cycle latency.
2. This would be a bug: two FUs writing the same phys reg means the rename unit allocated the same phys reg to two architectural destinations, violating the free-list invariant. The PRF would race-condition (last writer wins, but “last” is simulation-dependent). The rename unit must guarantee distinct allocations; this is a structural invariant we trust.
3. If PRF write fires but ROB mark-ready doesn’t: PRF has the new value (so dependent RSs can fire), but the ROB never marks the entry ready (so the instruction never commits, and the old phys reg is never freed). Eventually the free list runs out and the pipeline deadlocks. Both writes must always happen together.

Concept Check 3 (Widened RS)

1. Because issue_select_1 depends on issue_select_0 (it masks it out). Computing them in parallel would require knowing the answer before the computation, which is circular. The mask-and-encode pattern is fundamentally sequential at the logic

level. (In some real designs, parallel encoding is done with more complex circuitry like trees of priority encoders — but for small N, sequential is simpler.)

2. `issue_select_0` is one-hot pointing at the only ready RS. `issue_select_1` is all-zero (the mask removed the one ready bit). ALU0 receives the issue; ALU1 receives `valid=0` and does nothing. This is the correct behavior.
3. Because it requires combinational wakeup that checks an FU's in-flight output as well as the latched PRF. This makes the wakeup-select loop longer and harder to time. PRAVAH accepts the 1-cycle latency for simpler timing closure.

Concept Check 4 (Top-Level Integration)

1. Because the phys reg is allocated at dispatch, but the instruction doesn't *start* executing until its sources are ready. Between dispatch and execute, other RSs may have already snooped this phys reg's ready bit (looking for a producer). If the ready bit is still 1 (left over from a previous owner), the snooping RS will think the data is ready and read garbage. Clearing at dispatch ensures "ready=1" means "value is written by the current owner."
2. Because the PRF.ready is a combinational output of the PRF (just a wire from the `ready[]` array). The RS's snoop input is a combinational read. There's no flip-flop between them. When PRF.ready changes due to the clock edge, the RS sees it immediately at the start of the next cycle, without needing another clock.
3. Fetch keeps advancing PC even when dispatch is stalled. Instructions are read from memory, fed to decode, then to dispatch — but dispatch can't accept them. They're either dropped (PC advances past them without dispatch) or, worse, dispatch goes ahead and the rename unit's free list explodes. The `fe_stall` signal is the back-pressure that prevents this.

Concept Check 5 (Verilog)

1. Because if the same cycle's mark-ready and commit-check overlap, the commit logic (combinational, reading the registered ready bit) sees "not ready yet" but the mark-ready edge will set it. The instruction commits 1 cycle later. This 1-cycle latency is acceptable; PRAVAH's testbench tolerates it.
2. For interface consistency. If you later make the ALU pipelined (Week 7), it'll need a clock. Declaring the ports now means the instantiation in `pravah_top.v` doesn't change. Saves a refactor.
3. If both ROB slot A and slot B commit in the same cycle but rename only has 1 commit port, slot B's commit (the free of B's `old_phys_dest`) is lost. The phys reg is never returned to the free list. Eventually the free list runs out and the pipeline

stalls forever. Workaround for Week 6: only commit one slot per cycle. Better: widen rename to 2 commit ports.